Empirical Performance Analysis

Chris Kauffman

Last Updated:
Tue Mar 31 01:17:27 PM EDT 2026

Logistics

This Week

- Finish up Empirical Performance Analysis
- Start Shared Memory Architecture / Programming

Upcoming

A3 up now, due next week Tue, 3 problems

1. Analytic Performance of LU Factorization (medium)
2. HPC Toolkit Orientation (easy)
3. HPC Toolkit LU Empirical Performance (medium)

Performance of Programs, Serial and Parallel

- Parallel Programming is entertained for performance improvements: if it already runs "fast enough" serially, don't bother with parallelizing
- When seeking performance...
  1. Optimize the Serial code first
  2. *Then* attempt a parallel implementation
- If you're lucky, doing (1) may get to "fast enough"
- If not, your Parallel version will run all the faster
- "Fast Enough" is requires measurement so we'll review notions of time and speed

Exercise: Notions of Time in Computing

- In computing, several notions of **time** exist
- They are all related but fall into a couple categories
- What are important notions of time when evaluating program execution?
- Identify 2 separate kinds of "time" that are important
- Since we are dealing with parallelism, these two kinds of time are easier to identify

Answers: Notions of Time in Computing

CPU Time

- Total time processors executed to solve the problem
- Using 1 processor took 100 seconds: 100 CPU Seconds
- Using 4 processors tool 30 seconds: 120 CPU Seconds
- Correlates well with power usage / resource utilization etc.

Wall Time / Real Time

- How long humans waited for the solution
- Using 1 processor took 100 seconds: 100 seconds waiting
- Using 4 processors took 30 seconds: 30 seconds waiting
- Correlates with human goals like responsiveness ahead of time prediction

Both notions of time are important but one resorts to Parallel Programming to improve Wall Time

Measuring CPU and Wall Time in Serial Programs

- Measure CPU time with the standard `clock()` function; measure time difference and convert to seconds
- Measure Wall (real) time with `gettimeofday()` or related functions; fills struct with info on time of day

CPU Time

```
#include <time.h>

clock_t begin, end;
begin = clock(); // current cpu moment

do_something();

end = clock();   // later moment

double cpu_time =
  ((double) (end-begin)) / CLOCKS_PER_SEC;
```

`clock()` returns an arbitrary value on its own, requires subtraction to take the meaning of elapsed time

Real (Wall) Time

```
#include <sys/time.h>

struct timeval tv1, tv2;
gettimeofday(&tv1, NULL); // early time

do_something();

gettimeofday(&tv2, NULL); // later time

double wall_time =
  ((tv2.tv_sec-tv1.tv_sec)) +
  ((tv2.tv_usec-tv1.tv_usec) / 1000000.0);
```

`timeofday()` returns a meaningful value on its own but requires subtraction to take the meaning of elapsed time

The time Utility

```
# 'time prog args' reports 3 times for program runs by default
# - real: amount of "wall" clock time, how long you have to wait
# - user: CPU time used by program, sum of ALL threads in use
# - sys : amount of CPU time OS spends in system calls for program

>> time seq 10000000 > /dev/null          # print numbers in sequence
real    0m0.081s                          # real == user time
user    0m0.081s                          # 100% cpu utilization
sys     0m0.000s                          # 1 thread, few syscalls

>> time du ~ > /dev/null                  # check disk usage of home dir
real    0m2.012s                          # real >= user + sys
user    0m0.292s                          # 50% CPU utilization, lots of syscalls for I/O
sys     0m0.691s                          # I/O bound: blocking on hardware stalls

>> time ping -c 3 google.com > /dev/null # contact google.com 3 times
real    0m2.063s                          # real >>= user+sys time
user    0m0.003s                          # low cpu utilization
sys     0m0.007s                          # lots of blocking on network

>> time make > /dev/null                  # make with 1 thread
real    0m0.453s                          # real == user+sys time
user    0m0.364s                          # ~100% cpu utilization
sys     0m0.089s                          # syscalls for I/O but not I/O bound

>> time make -j 4 > /dev/null             # make with 4 "jobs" (threads/processes)
real    0m0.176s                          # real <= user+sys
user    0m0.499s                          # syscalls for I/O and coordination
sys     0m0.111s                          # parallel execution gives SPEEDUP!
```

Hardware Counters for Performance Analysis

- CPUs usually have **hardware counters** in them that measure events like cache hits/misses, floating point performance, etc.
- The Linux `perf` tool is a general tool that can access some of these counters for serial programs and help reveal things like memory performance

perf on sumR vs sumC

Below is a run of summing a matrix by either Iterating Across Rows (sumR) or Iterating Down Columns (sumC). The hardware counters clearly show that for Row-Major language like C, Row-wise access is more performant.

```
>> perf stat $perfopts ./matrix_timing 8000 4000 row    ## RUN sumR ROW SUMMING
sumR: 1227611136 row-wise CPU time: 0.019 sec, Wall time: 0.019
Performance counter stats for './matrix_timing 8000 4000 row':       %SAMPLED
135,161,407  cycles:u                                                (45.27%)
417,889,646  instructions:u          #  3.09  insn per cycle         (56.22%)
 56,413,529  L1-dcache-loads:u                                       (55.96%)
  3,843,602  L1-dcache-load-misses:u #  6.81% of all L1-dcache hits  (50.41%)
 28,153,429  L1-dcache-stores:u                                      (47.42%)
        125  L1-icache-load-misses:u                                 (44.77%)
  3,473,211  cache-references:u      # last level of cache           (56.22%)
  1,161,006  cache-misses:u          # 33.427 % of all cache refs    (56.22%)

>> perf stat $perfopts ./matrix_timing 8000 4000 col   # RUN sumC COLUMN SUMMING
sumC: 1227611136 col-wise CPU time: 0.086 sec, Wall time: 0.086
Performance counter stats for './matrix_timing 8000 4000 col':        %SAMPLED
372,203,024   cycles:u                                                (40.60%)
404,821,793   instructions:u          #  1.09  insn per cycle         (57.23%)
 61,990,626   L1-dcache-loads:u                                       (60.21%)
 39,281,370   L1-dcache-load-misses:u # 63.37% of all L1-dcache hits  (45.66%)
 23,886,332   L1-dcache-stores:u                                      (43.24%)
      2,486   L1-icache-load-misses:u                                 (40.82%)
 32,582,656   cache-references:u      # last level of cache           (59.38%)
  1,894,514   cache-misses:u          #  5.814 % of all cache refs    (60.38%)
```

`perf` stats for sumR vs sumC, what's striking?

Observations

- Similar number of instructions between row/col versions
- `#cycles` lower for row version → higher `insn per cycle`
- **L1-dcache-misses**: marked difference between row/col version
- **Last Level Cache Refs** : many, many more in col version
- Col version: much time spent waiting for memory system to feed in data to the processor

Notes

- The right-side percentages like (50.41%) indicate how much of the time this feature is measured; some items can't be monitored all the time.
- Specific `perf` invocation is in `10-emipirical-performance-analysis-code/measure-cache.s`

Time in Parallel Programs

- `MPI_Wtime()` provides a way to measure Wall Time in MPI programs
- Calling convention are similar to `clock()` for serial programs but gives the Wall time
- Typically only the Root Processor would report this
- Why not CPU time? Why only the Root?

```
double start, end;
double phase1, phase2, phase3;
start = MPI_Wtime();
// ... phase1 code ...
end = MPI_Wtime();
phase1 = end - start;

start = MPI_Wtime();
// ... phase2 ...
end = MPI_Wtime();
phase2 = end - start;

start = MPI_Wtime();
// ... phase3 ...
end = MPI_Wtime();
phase3 = end - start;

if(myrank==0){
  printf("Phase 1: %5.2f s\n",phase1);
  printf("Phase 2: %5.2f s\n",phase2);
  printf("Phase 3: %5.2f s\n",phase3);
}
// Phase 1:  2.45 s
// Phase 2: 11.79 s
// Phase 3:  4.37 s
```

Tracing and Profiling Code

- **Tracing** means to record certain events (like function calls) in a program run with enter/leave timestamps
  - Often accomplished by *instrumenting* the code during compilation
  - A few assembly instructions are added at the start/end of each function that record entry/exit times for them
  - When the program is run, events are counted then dumped into a data file
- **Profiling** means to gather information about where a program is spending its time in aggregate
  - Often accomplished by a monitoring tool
  - Every so often (100 milliseconds) the program is paused by the monitoring tool
  - The monitoring tool examines the program state (Instruction Pointer, Stack Pointer) and determines what is being done
  - Records adds sample to collection, dumps file at the end
- There is significant overlap between these ideas a
- Lots of tools exist for both Serial and Parallel Programs that provide Tracing, Profiling, or Both

Profiling of Serial Codes with gprof

- gprof is a standard performance analysis tool on Linux systems, interfaces with gcc
- Compile with profiling options: gcc -pg
- Run code to produce data file, format profile data with gprof

```
# Compile
# -pg : instrument code for profiling
# -Og : debug level optimization
# -g  : include debug symbols
>> gcc -pg -g -Og -o unroll unroll.c

>> ls
unroll  unroll.c

>> ./unroll 1000000000
sum_rangeA: 2.9401e-01 secs
sum_rangeB: 5.3164e-01 secs
sum_rangeC: 2.6574e-01 secs

# gmon.out now created with timing info
>> ls
gmon.out  unroll  unroll.c

>> file gmon.out
gmon.out: GNU prof performance data

>> gprof -b unroll
... output on next slide ...
```

gprof output for unroll

```
>> gprof -b unroll
Flat profile:
Each sample counts as 0.01 seconds.
  %   cumulative   self              self     total
 time   seconds   seconds    calls  ms/call  ms/call  name
 50.38      0.54     0.54        1   544.06   544.06  sum_rangeB
 26.12      0.83     0.28        1   282.11   282.11  sum_rangeA
 24.26      1.09     0.26        1   261.95   261.95  sum_rangeC

                        Call graph
index % time    self  children    called     name
[1]    100.0    0.00    1.09                 main [1]
                0.54    0.00       1/1           sum_rangeB [2]
                0.28    0.00       1/1           sum_rangeA [3]
                0.26    0.00       1/1           sum_rangeC [4]
-----------------------------------------------
                0.54    0.00       1/1           main [1]
[2]     50.0    0.54    0.00       1         sum_rangeB [2]
-----------------------------------------------
                0.28    0.00       1/1           main [1]
[3]     25.9    0.28    0.00       1         sum_rangeA [3]
-----------------------------------------------
                0.26    0.00       1/1           main [1]
[4]     24.1    0.26    0.00       1         sum_rangeC [4]
-----------------------------------------------
```

Typical Usage of Trace/Profile Information

- Optimize **hot spots**, functions or sections of code which are particularly expensive
- `gprof` gives good insight on functions that are expensive BUT less informative which parts of functions need attention (HPC Toolkit is better at this)

Sample Application: `dictionary.c`

- Counts word frequencies in a text file, identifies most frequent word, longest word, expandable to count most frequent 2-grams, 3-grams, ... N-grams (though runs much longer)
- 700 lines of code, ~28 functions, myriad command line options, minimal commenting
- How would one determine the best place to focus optimization effort?

gprof Example: Dictionary Application

```
>> ./dictionary < craft-67.txt
... Total time = 0.829561 seconds
>> gprof -b dictionary
  %   cumulative   self              self     total
 time   seconds   seconds    calls  ms/call  ms/call  name
 50.07      0.18     0.18        1   180.25   180.25  sort_words
 19.47      0.25     0.07   463016     0.00     0.00  find_ele_rec
 13.91      0.30     0.05  2862749     0.00     0.00  Strlen
  8.34      0.33     0.03   463016     0.00     0.00  lower1
  2.78      0.34     0.01   463017     0.00     0.00  get_token
  2.78      0.35     0.01   463016     0.00     0.00  h_mod
  2.78      0.36     0.01    20451     0.00     0.00  save_string
  0.00      0.36     0.00   463017     0.00     0.00  get_word
  0.00      0.36     0.00   463016     0.00     0.00  insert_string
  0.00      0.36     0.00    20451     0.00     0.00  new_ele
  0.00      0.36     0.00        7     0.00     0.00  add_int_option
  0.00      0.36     0.00        1     0.00     0.00  add_string_option
  0.00      0.36     0.00        1     0.00     0.00  init_token
  0.00      0.36     0.00        1     0.00     0.00  new_table
  0.00      0.36     0.00        1     0.00     0.00  parse_options
  0.00      0.36     0.00        1     0.00     0.00  show_options
  0.00      0.36     0.00        1     0.00   360.50  word_freq
```

gprof Example Cont'd: Dictionary Application

```
>> ./dictionary < craft-67.txt     ## After upgrading sort_words() to qsort()
... Total time = 0.624172 seconds
>> gprof -b dictionary
  %   cumulative   self              self     total
 time   seconds   seconds    calls  ms/call  ms/call  name
 60.08      0.12     0.12   463016     0.00     0.00  find_ele_rec
 15.02      0.15     0.03  2862749     0.00     0.00  Strlen
 10.01      0.17     0.02   463016     0.00     0.00  lower1
  5.01      0.18     0.01   463017     0.00     0.00  get_token
  5.01      0.19     0.01   463016     0.00     0.00  h_mod
  5.01      0.20     0.01    20451     0.00     0.00  save_string
  0.00      0.20     0.00   463017     0.00     0.00  get_word
  0.00      0.20     0.00   463016     0.00     0.00  insert_string
  0.00      0.20     0.00    20451     0.00     0.00  new_ele
  0.00      0.20     0.00        8     0.00     0.00  match_length
  0.00      0.20     0.00        7     0.00     0.00  add_int_option
  0.00      0.20     0.00        1     0.00     0.00  add_string_option
  0.00      0.20     0.00        1     0.00     0.00  find_option
  0.00      0.20     0.00        1     0.00     0.00  init_token
  0.00      0.20     0.00        1     0.00     0.00  new_table
  0.00      0.20     0.00        1     0.00     0.00  parse_options
  0.00      0.20     0.00        1     0.00     0.00  show_options
  0.00      0.20     0.00        1     0.00     0.00  sort_words ** was 0.18 **
  0.00      0.20     0.00        1     0.00   200.28  word_freq
```

HPC Toolkit Profiling

HPC Toolkit is great: it's like `gprof` for parallel programs!
– Abhinav Bhatele

- HPC folks are of course interested in optimizing programs and have many tools to analyze program runs
- Among the better and more standardized is HPC Toolkit (https://hpctoolkit.org/) which allows profiling and tracing of parallel codes AND provides a nice analysis tool
- Generates both Profile and Trace information on runs per request
- Performance measurement data stored in a directory for later processing
- Provides a nice Viewer for analysis of data
- Can only Profile/Trace code running on Linux (e.g. Zaratan) but can View profile data on most OSes

HPC Toolkit Profiling / Tracing

- No special requirements for building programs EXCEPT including compiler debug info via -g
- HPC Toolkit must be installed on the run system; on Zaratan accessible via

```
>> module load hpctoolkit
```

- Alters PATH to access HPC Toolkit commands
- Profile and/or Trace code via

```
## Profile Only a normal program
>> hpcrun    myprogram arg1 arg2 arg3 ...

## Profile and Trace normal program
>> hpcrun -t myprogram arg1 arg2 arg3 ...

## Profile Only MPI Run
>> mpirun -np 8 hpcrun    myprogram arg1 arg2 arg3 ...

## Profile and Trace MPI Run
>> mpirun -np 8 hpcrun -t myprogram arg1 arg2 arg3 ...
```

- The -t option is important for Trace Information (behavior over time)

Data Post Processing in HPC Toolkit

- Data from HPC Toolkit is placed in a directory named something like `hpctoolkit-myprog-measurements-1234567`
- Requires post processing to tie measurements to parts of the program
- Post process with these two commands in order

```
# analyzes binary program relate it to source code INCLUDING nested
# loop structure to reveal hot spots
>> hpcstruct hpctoolkit-myprog-measurements-1234567
...

# relates measurement events to original source to produce a databse
# of function calls / code execution over time
>> hpcprof   hpctoolkit-myprog-measurements-1234567
...
```

- Post processed data is now in

  `hpctoolkit-myprog-database-1234567`

- For very large post-processing, there is also an `hpcprof-mpi` version to act in parallel but we don't need this

Sidebar: Interactive Slurm Jobs

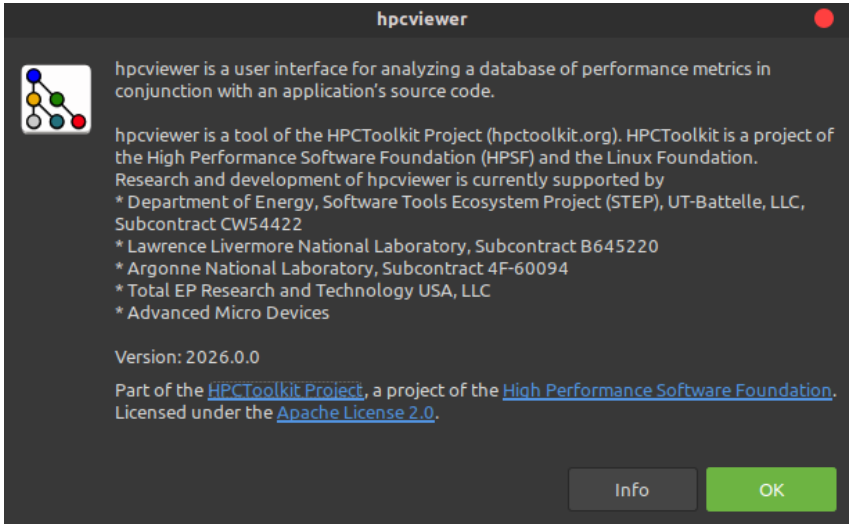
- Can be a bit tedious to set up short, experimental work in a batch job to be submitted via `sbatch`
- Instead, an **interactive computer job** is handy and facilitated by **`sinteractive`**, the SLURM system's command to request a compute node for interactive use
- Acquiring the resources may take a moment or two (but Zaratan has a LOT of resources)
- Sample session with `sinteractive`:

```
profk@login-3 [~]% sinteractive -c 8 -t 20 -a cmsc416-class
salloc: Pending job allocation 18741203
salloc: job 18741203 queued and waiting for resources
salloc: job 18741203 has been allocated resources
salloc: Granted job allocation 18741203
salloc: Nodes compute-a5-9 are ready for job

profk@compute-a5-9 [~]% mpirun -np 8 hpcrun -t myprogram ...

profk@compute-a5-9 [~]% exit
salloc: Relinquishing job allocation 18741203

profk@login-3 [~]%
```

HPC Viewer for Interactive Analysis

After obtaining and post-processing run data, the `hpctoolkit-...-database-...` directory can be analyzed via **`hpcviewer`**, GUI program that shows

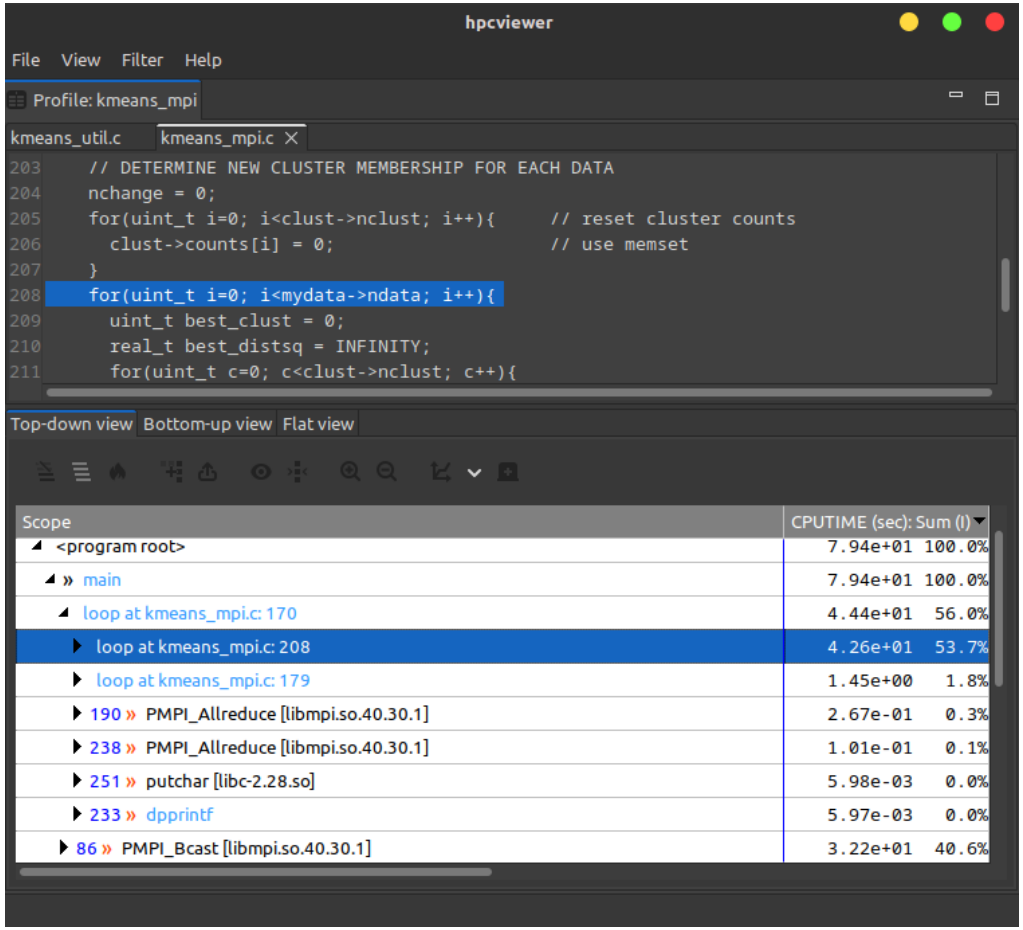
- Profile: CPU time spent across all CPUs in different functions and in different nested loops
- Trace (with `hpcrun -t`): Activity over time for all processors with colors corresponding to activity (call stack)



HPC Viewer Sample: Profile

- Expand to view fraction of **total CPU time** spent in parts of program
- Identify hot spots worth optimizing
- Click to show source code locations

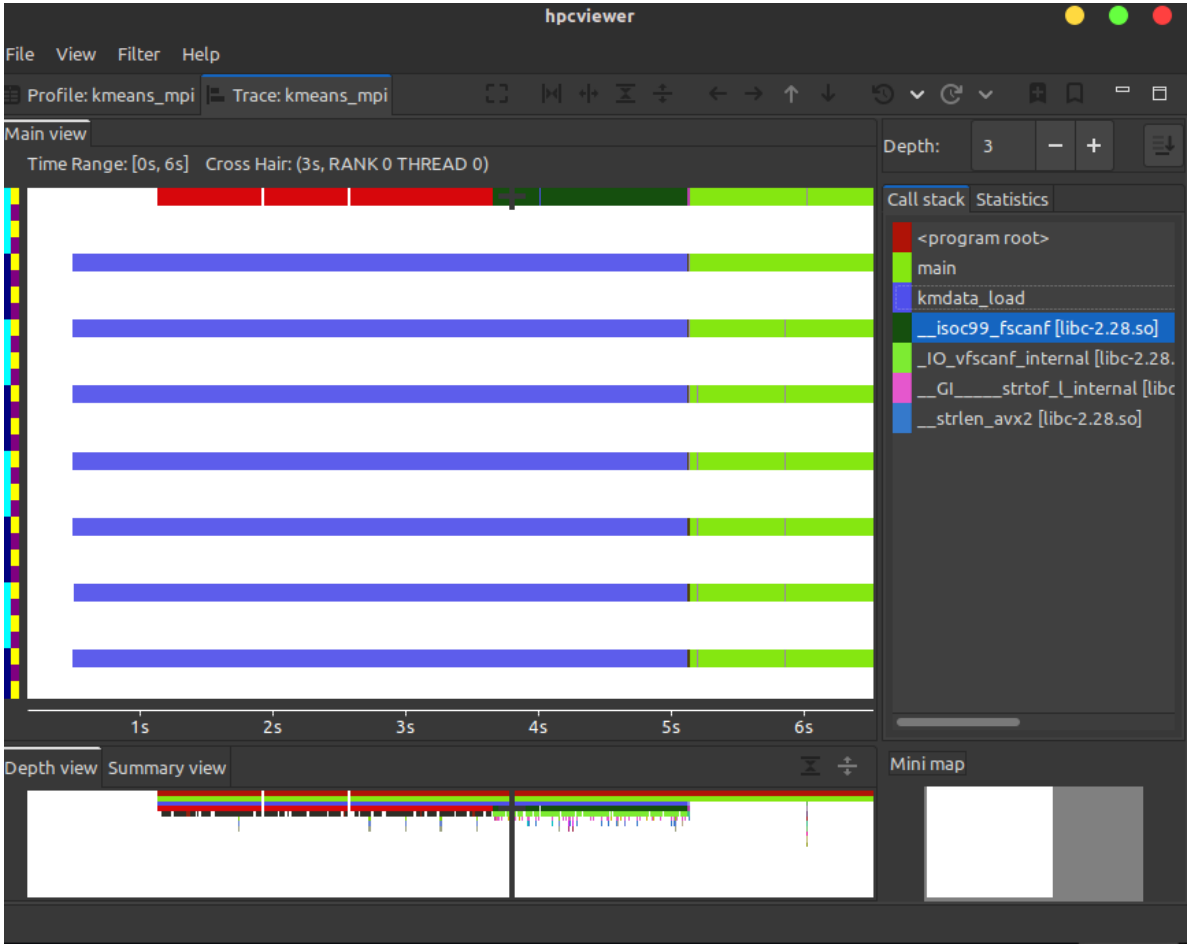
Caution: profile on its on may be misleading as it sums CPU time and MPI calls may involve polling (busy waiting)



`kmeans_mpi` run with 8 procs

HPC Viewer Sample: Trace

- Bars represent each processor (thread)
- Color shows function activity
- Click to show call stack for thread
- Limit Stack Call Depth for better insight
- View whole run via default view (total runtime)
- Zoom in / out for individual sections



`kmeans_mpi` run with 8 procs

Tricks in HPC Viewer

- Trace view is very useful for analyzing behavior over time: shows total runtime and where different procs spent portions of time
- Colors can be a little weird at first: **adjust the call Depth down** to agglomerate call stacks at a lower granularity
- Keep in mind that MPI communication involves polling: CPU spins in procs waiting for communication

Hatchet: Programmatic Analysis of Parallel Profiles

- HPC Toolkit provides a rich set of information about program runs
- Its HPC Viewer is good for humans to do interactive analysis of data
- No tooling is provided to do scripted / command line analysis
- Enter UMD's Hatchet, a Python library for doing programmatic analysis of the profile databases like what HPC Toolkit provides
- If in want of a scripted experience, consider this as an alternative